

Minecraft 建筑生成多智能体系统：从中文需求到可执行数据包

《大语言模型与信息决策》课程项目书面报告

龙想 2300011196；石宇宸 2300011051

2026 年 6 月

<https://github.com/CityC196/Minecraft-Constructing-Agents>

本报告强调项目构思中的关键修改：从直接生成方块坐标，转向 LLM 语义 JSON + 本地确定性几何引擎，并进一步加入模板语料学习和质量闭环。

摘要

本项目面向 Minecraft Java 1.21 的建筑自动生成任务，尝试把中文自然语言建房需求转换为可直接安装到游戏世界的数据包。项目的核心取舍是：不让大语言模型直接输出具体 XYZ 方块坐标，而是让其生成建筑风格、材料、体块、房间拓扑和设计意图等高层语义 JSON；随后由本地 JavaScript 几何引擎完成 CSG 体块生成、BSP 房间切分、A* 门洞和楼梯连通、室内装饰落块、命令压缩与校验。

在实现过程中，项目从最初的单一建筑智能体逐步迭代为多智能体流水线，并加入模板语料学习、设计法则蒸馏、审美审计、自动修复和候选择优。当前系统可以在没有 API key 的 mock 模式下完整运行，生成 blueprint.json、architect_datapack、preview.html 和 run_report.md。仓库基线测试为 173 项通过；加入课程提交文档测试后，当前全量测试为 176 项通过、0 项失败。

关键词：大语言模型智能体；Minecraft；多智能体系统；CSG；BSP；A*；数据包

1. 选题动机：开放建筑任务中的不确定性

Minecraft 建筑生成看似只是“把房子搭出来”，但实际同时包含语义理解、空间规划、材料选择、几何约束、连通性、审美风格和游戏版本兼容等问题。用户的一句中文需求可能只写“建一个欧式大房子”，但系统需要进一步判断尺度、层数、入口、房间、屋顶、窗、路径、装饰和数据包输出方式。

如果直接要求大模型生成每一个方块坐标，结果通常难以稳定复现：坐标可能越界，材料 ID 可能不存在，房间可能不连通，楼梯或门洞可能缺失，生成命令也可能超过 Minecraft 限制。基于这个观察，我们把项目目标调整为“让 LLM 做语义决策，让本地程序做确定性落地”。这个改动是后续所有架构设计的出发点。

- 课程相关性：项目围绕大模型智能体与多智能体协作展开，而不是普通游戏脚本。
- 任务完整性：从中文输入到 Minecraft 数据包输出形成闭环。
- 可复现性：mock 模式不依赖 token 或外部 API，便于评阅。

2. 总体方案：LLM 语义智能体 + 本地几何引擎

项目采用混合式架构。ArchitectAgent、PlannerAgent、CreativeDesignAgent 等智能体只生成高层 JSON，描述风格、材料、体块、房间拓扑、立面节奏和场地语义。CSGBuilder、BSPPartitioner、AStarPathfinder 等本地模块再把语义描述变成合法方块网格。最后由 DecoratorAgent、QA、Repair 和 Optimizer 完成室内装饰、约束修复、质量检查和命令压缩。

这一架构把大模型的创造力保留在适合它的抽象层，同时把确定性、安全性和 Minecraft 版本兼容交给程序。它也让系统可以在 LLM 不可用时退回本地规则兜底，保持一个稳定演示和提交版本。

层次	主要模块	作用
语义层	ArchitectAgent / PlannerAgent	生成外壳语义、材料意图、房间拓扑和连通要求
知识层	TemplateKnowledgeAgent	检索模板案例、设计法则和非复制约束
几何层	CSG / BSP / A*	生成体块、房间、门洞、楼梯和可通行路径

导出层	Decorator / QA / Optimizer	写入内饰、校验质量、压缩命令并导出数据包
-----	----------------------------	----------------------

3. 多智能体流程

当前主流程位于 `src/construction/workflow.js`，围绕 `construction_method_v1` 展开。用户输入首先被解析为 `buildSpec`；随后 `ArchitectAgent` 生成建筑外壳语义，`MaterialPaletteAgent` 校验并扩展 Minecraft 1.21.1 方块材料，`PlannerAgent` 生成房间节点和连通边，`CreativeDesignAgent` 决定体块变体、屋顶、立面和场地策略。

后半段由本地几何与装饰模块完成。`CSGBuilder` 生成空心外壳，`BSPPartitioner` 切分室内房间，`AStarPathfinder` 打通门洞和楼梯。`InteriorDetailAgent` 与 `DecoratorAgent` 负责家具、灯光、植物、地毯和功能台面。最后，`BlueprintQAAgent`、`ConstraintRepairAgent`、`TemplateLawAutoRepairAgent` 和 `BlueprintOptimizerAgent` 检查可达性、材料合法性、模板法则覆盖和命令数量。

- LLM 只输出 JSON：降低坐标幻觉和非法命令风险。
- 本地几何保证结构：外壳、房间、门洞、楼梯由程序计算。
- 导出物完整：每次运行生成 `blueprint.json`、`datapack`、`raw_build.mcfuction`、`preview.html` 和 `run_report.md`。

4. 几何落地与 Minecraft 数据包

几何层是项目中工作量较大的部分。`CSGBuilder` 将主体、侧翼、门廊、塔楼等语义体块合并为稀疏 voxel 网格，并抽取外表面、楼板、屋顶和窗。`BSPPartitioner` 根据房间权重和楼层功能切分内部空间，`AStarPathfinder` 根据拓扑关系生成门洞和楼梯。这样可以避免 LLM 直接生成坐标时常见的穿墙、断路和房间重叠问题。

导出层会把方块网格转换成 Minecraft 1.21 数据包。数据包入口为 `data/architect/function/run.mcfuction`，玩家在游戏中执行 `/reload` 后，再执行 `/function architect:run` 即可触发 `clear + build`。系统还会输出 `raw_build.mcfuction` 和 `preview.html`，方便不打开游戏时检查结构。

以本地样例 `out/2026-06-19-145532212` 为例，`run_report.md` 记录的建筑尺寸为 43 x 16 x 57，房间可达性为 14/14，QA 为 9/9 项通过，装饰数量为 819 个。Exporter 将朴素命令 2692 条压缩到 1251 条，压缩倍率约 2.15x。这些统计说明系统不只是生成一段文本，而是在输出前做了结构、材料、连通和命令层面的检查。

输出文件	用途
<code>blueprint.json</code>	保存 prompt、seed、Agent 输出、几何统计、房间、装饰和校验结果
<code>architect_datapack/</code>	可复制到 Minecraft 世界 <code>datapacks</code> 目录的完整数据包
<code>preview.html</code>	本地浏览器中的静态平面和阶段预览
<code>run_report.md</code>	记录本次生成流程、LLM 使用情况、几何统计和使用步骤

5. 模板语料学习：把参考建筑变成设计语法

项目后期加入了 mc_templates/ 下的本地 schematic 语料，共 64 个模板，覆盖住宅、城堡、塔楼、寺庙、竞技场和公共建筑等类型。我们没有把模板当成方块级复制库，而是把它们分析为可迁移的设计语法：场地入口、前景花园、体块轮廓、屋顶语言、立面深度和室内场景。

模板分析会生成案例库、语义条款、设计法则和检索索引。生成时，TemplateKnowledgeAgent 结合用户 prompt 检索相关参考，CreativeDesignAgent 和 Site/Roof/Facade/Interior 等模块再使用这些语法。为降低抄袭风险，系统记录 source_fusion_policy，要求多来源融合、控制单一来源占比，并将大型纪念性模板归一化到住宅尺度。

这一阶段也促使我们修改了项目目标：系统不再只追求“有墙、有屋顶”，而是尝试让建筑拥有场地关系、入口序列、立面深度、屋顶语言和室内生活场景。docs/template-assimilation-plan.md 中记录的最终回归结果显示，36 个模板吸收 prompt 均成功生成，平均 template audit 和 law coverage 达到 100%。自动评分不能完全代表视觉审美，但它能证明模板知识链路已经接入主流程。

- 模板数量：64 个本地 schematic，导入错误为 0。
- 设计法则：包括 site、massing、roof、facade 和 interior 多类语义。
- 非复制控制：强调借鉴结构语法，而非 1:1 复制模板。

6. 项目迭代与构思修改

这个项目的重点不只是最终代码量，而是我们在构思中不断修改问题定义。最开始的目标偏向“让智能体建一个 Minecraft 房子”，后来发现如果只是堆 prompt 或直接生成命令，既不稳定，也不容易体现智能体系统的结构。于是项目逐步转向多智能体分工和确定性几何落地。

Git 历史能看到几个关键阶段：5 月 28 日建立初始 Minecraft architect agent，并加入自动安装世界与建造模式；6 月 2 日拆分蓝图生成子 Agent；6 月 9 日加入语义规划层和多智能体架构；6 月 17 日完成更大规模的 construction_method_v1；6 月 18 日引入模板语料和 schematic 分析；6 月 19 日加入设计法则、审美审计、自动修复、候选择优和最终 README 收束。

阶段	主要变化	背后的取舍
初始版本	自然语言到建筑输出	先建立可运行闭环
多 Agent 拆分	拆分 Designer/Planner/Critic/Repair 等职责	避免单 prompt 承担全部逻辑
语义规划层	LLM 输出 JSON 而非坐标	提高稳定性和可验证性
确定性几何	CSG、BSP、A* 本地落地	用程序保证结构合法
模板学习	案例库、设计法则、审美审计	让建筑更像“有风格的设计”而非空壳

7. 实验与运行结果

截至本报告生成时，项目全量测试为 176 项通过、0 项失败，其中包括 3 项课程提交文档测试；原项目功能基线为 173 项通过、0 项失败。测试覆盖 Architect fallback、Planner、CSG、BSP、A*、Decorator、模板法则、候选择优、LLM provider、材料目录、可视化输出等模块。

本地样例输出目录 out/2026-06-19-145532212/ 包含 blueprint.json、architect_datapack、raw_build.mcfuction、preview.html 和 run_report.md。该样例记录了 prompt“建造一个木屋别墅”、LLM 调用状态、几何统计、装饰数量、模板法则覆盖率、QA 状态和 Minecraft 使用步骤。

为了避免现场演示的不确定性，项目把每次生成的关键证据写入 run_report.md。报告会记录本次是否调用 LLM、seed 来源、体块列表、材料角色、房间节点、场地策略、模板审计分数、命令数量和 Minecraft 操作步骤。这样即使不打开游戏，也可以追踪一次生成是否真实发生。

- 测试命令：npm test。
- mock 运行命令：npm start -- --mode mock "建一个欧式大房子"。
- Minecraft 命令：/reload，然后 /function architect:run。
- 运行结果截图补充位：可后续加入 preview.html 截图或游戏内建筑截图。

样例指标	数值	说明
建筑尺寸	43 x 16 x 57	来自 out/2026-06-19-145532212/run_report.md
房间连通	14/14	入口可达房间全部通过
QA	9/9	结构、材料、入口和装饰等检查通过
命令压缩	2692 -> 1251	减少函数体积，便于数据包执行

8. 整体性、创新性与课程要求对齐

课程评分强调工作量、整体性和创新性。本项目在整体性上围绕一个明确问题展开：如何把开放中文建筑需求稳定落地为 Minecraft 可执行数据包。所有 Agent、几何模块、模板语料和评测脚本都服务于这个目标，而不是把多个无关 skill 拼在一起。

创新性主要体现在三个方面。第一，LLM 与本地几何的边界划分比较清楚，避免了直接坐标生成的不可控。第二，模板语料被抽象为设计语法和审美法则，而不是简单复制。第三，项目保留 mock 兜底和测试闭环，使没有 token 或现场网络时仍能完整演示。

- 工作量：多智能体、几何引擎、模板分析、导出、测试和文档完整覆盖。
- 整体性：所有模块围绕自然语言到 datapack 的主流程协作。
- 创新性：将建筑参考案例转化为可检索、可修复、可审计的语法层。

9. 不足与后续方向

当前系统已经实现从中文 prompt 到 Minecraft 数据包的闭环，但它仍不是 Mineflayer 实时机器人，也不模拟玩家在服务器中逐块放置。GDMC HTTP 客户端已经保留，但主流程仍以数据包导出为主。另一方面，自动审美评分可以发现法则覆盖问题，却不能完全替代玩家视角下的尺度、镜头、路径和 block readability 检查。

后续可以继续扩展三个方向：第一，加入更可靠的视觉相似性和非复制检测；第二，补充更多真实游戏截图与人工审美评分样本；第三，将 GDMC 或 Mineflayer 执行路径与现有 datapack 路径整合，让系统既能离线导出，又能在游戏内交互式部署。

10. 工作分工与 AI 辅助说明

本项目由龙想、石宇宸组成小组完成。小组共同参与选题确定、项目边界设定、系统架构取舍、运行调试、结果确认和最终提交材料整理。代码仓库中的提交记录反映了从初始 agent 到多智能体、语义规划、模板学习和最终提交材料的连续迭代。

项目合理使用 AI 工具辅助开发和文档整理。AI 辅助主要用于代码修改建议、文档初稿整理、报告润色、测试清单和网页排版检查；项目方向、架构取舍、运行验证、测试执行、结果是否采纳和最终整合由小组完成。报告和网站中不会伪造运行结果，缺少截图的位置均标记为真实截图补充位。

11. 提交材料与补图说明

最终提交建议包含三类材料：第一，GitHub 仓库链接，用于查看源码、测试和文档；第二，本报告 PDF，用于教学网书面报告提交；第三，docs/index.html 展示页和 SUBMISSION.md 提交清单，用于快速理解项目结构与复现方式。由于课程要求不得伪造运行结果，当前缺少的 Minecraft 游戏内截图以“真实截图补充位”呈现。

如果后续需要补充截图，可以先运行 `npm start -- --mode mock` "建一个欧式大房子"，打开输出目录中的 `preview.html` 截图；也可以将 `architect_datapack` 复制到 Minecraft 世界 `datapacks` 目录，进入游戏执行 `/reload` 与 `/function architect:run` 后截图。建议补充 2 到 3 张图：`preview.html` 总览、游戏内外观、游戏内室内或入口路径。

- 不要把 `.env`、API key、`out/` 大型生成目录或课程 PDF 提交到源码仓库。
- 报告 PDF 可以作为教学网附件提交；源码仓库保持可运行和可复现。
- 补图时只使用真实运行结果，不使用生成式图片冒充 Minecraft 输出。

12. 结论

Minecraft Constructing Agents 通过多智能体语义规划和本地确定性几何引擎，把一个开放的中文建筑需求转化为可复现、可校验、可安装的数据包输出。项目的价值不在于某一次生成结果一定完美，而在于它把大模型智能体的创造性、本地算法的可靠性、模板语料的风格经验和工程测试闭环整合到同一条流水线中；从课程项目角度看，最重要的探索是从直接生成坐标，逐步改为多 Agent 分工、语义 JSON、确定性几何和模板质量闭环。